



Busqueda de factores RSA compartidos en los registros de Transparencia de Certificados

Un enfoque basado en el algoritmo Binary Tree Batch GCD

David Quinzanos Saiz

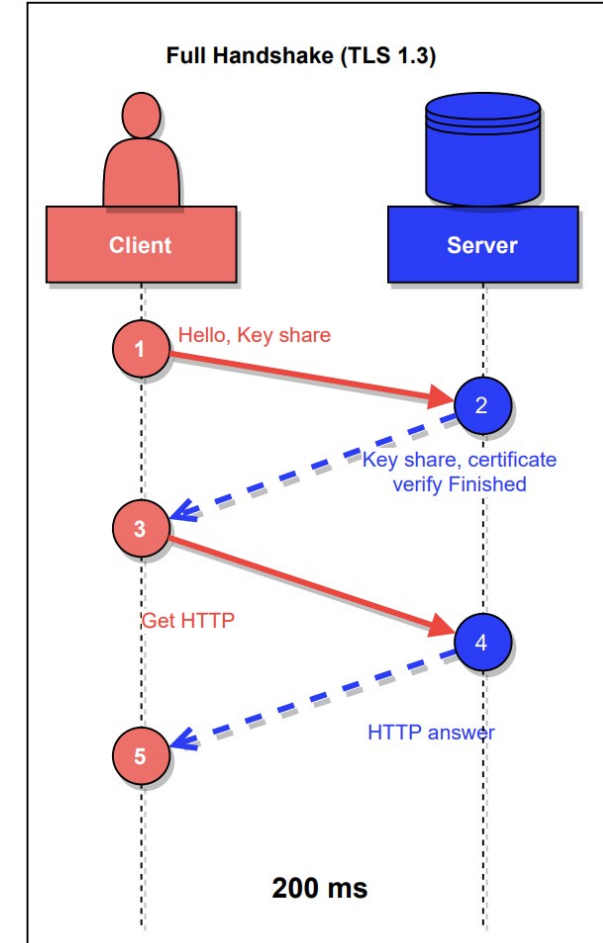
Codirectores: Ana I. Gomez Perez - Domingo Gomez Perez

Master Interuniversitario (UC-UIMP) en Data Science - Junio de 2026

● MOTIVACION

La confianza en la Web descansa en los certificados

- HTTPS/TLS protege la comunicacion entre el navegador y el servidor.
- El navegador confia en un certificado X.509 firmado por una autoridad certificadora (CA).
- El certificado vincula una identidad (dominio) con una clave publica.
- Certificate Transparency obliga a publicar los certificados en logs publicos y auditables.



Intercambio TLS (cliente-servidor)

RSA y la importancia de los primos

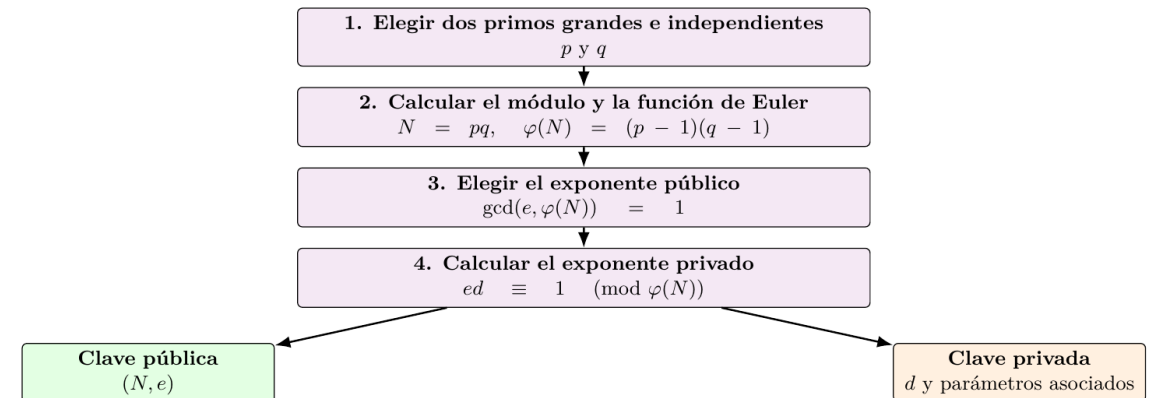
$$N = p \cdot q$$

Clave pública: (N, e)

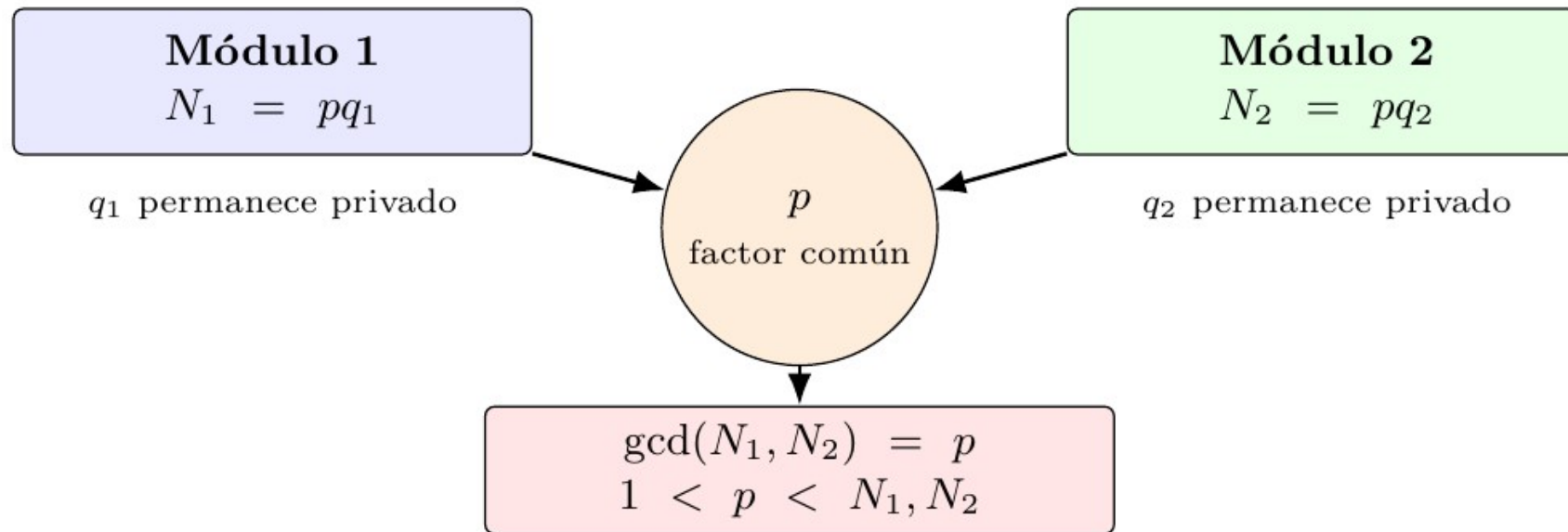
Clave privada: $d = e^{-1} \pmod{\phi(N)}$

asociada a la factorización de N

- p y q deben ser primos grandes, generados de forma aleatoria e independiente.
- La seguridad descansa en la dificultad de factorizar N .



Vulnerabilidad: factores primos compartidos



Si dos modulos comparten un primo, **$\gcd(N_1, N_2) = p$** recupera el factor y compromete ambas claves, sin factorizar.

El problema de escala

$$M(M-1)/2$$

comparaciones para M modulos

54.275

modulos reales analizados ->

1.472.860.675

comparaciones por fuerza bruta

- Calcular un GCD aislado es barato...
- ...pero comparar todos los pares crece de forma cuadratica.
- Inviabile para millones de claves.
- Necesitamos un enfoque global: batch GCD.

La tesis de Vage: auditar CT logs

- Vage (2022, Univ. de Bergen) toma los CT logs como fuente de claves RSA reales.
- Demuestra que auditar esas claves tiene sentido practico y sigue dando resultados.
- Deja abierta la idea clave: mejorar el algoritmo permitiria analisis mucho mayores.

159.377.664

claves RSA-2048 de CT logs

8

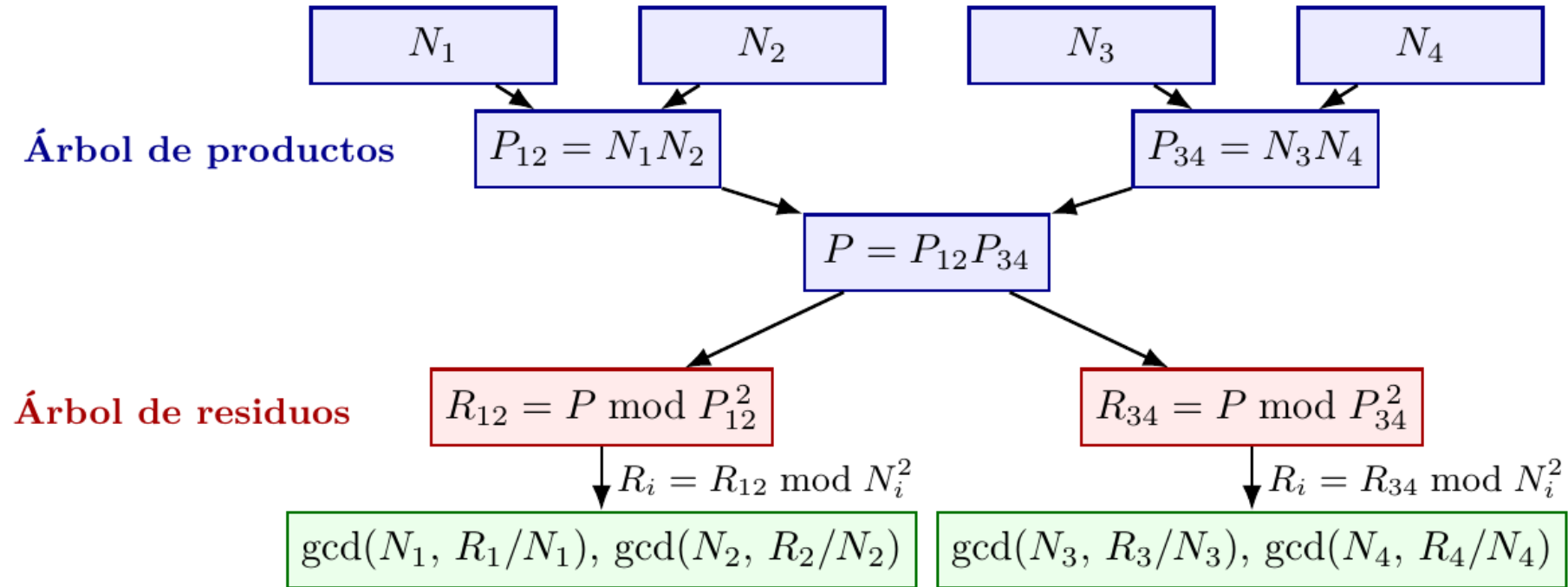
claves vulnerables (ZeroSSL)

355

claves unicas factorizadas (de >700.000)

Fuente: Vage (2022), Master's thesis, University of Bergen.

Metodo clasico: product tree + remainder tree



Modulos -> product tree -> producto global P -> remainder tree -> GCD finales. Evita la comparacion par a par reorganizando el calculo globalmente.

Limitaciones del metodo clasico al escalar

- Los enteros intermedios en la cima del arbol son gigantescos.
- Alto coste en tiempo y, sobre todo, en memoria.
- Obliga a dividir la coleccion en lotes y cruzarlos entre si.

13

lotes de ~12 M de claves

~180 GB

memoria RAM

~1 TB

almacenamiento en disco

~88 h

con 30 nucleos

Ejecucion principal descrita por Vage (2022).

De Vage a Pelofske: donde encaja este TFM

Vage (2022)

- CT logs como fuente de claves RSA reales
- batch GCD clasico
- Problema aplicado



Pelofske (2024)

- Binary Tree Batch GCD
- Mejora algoritmica
- Evita el remainder tree



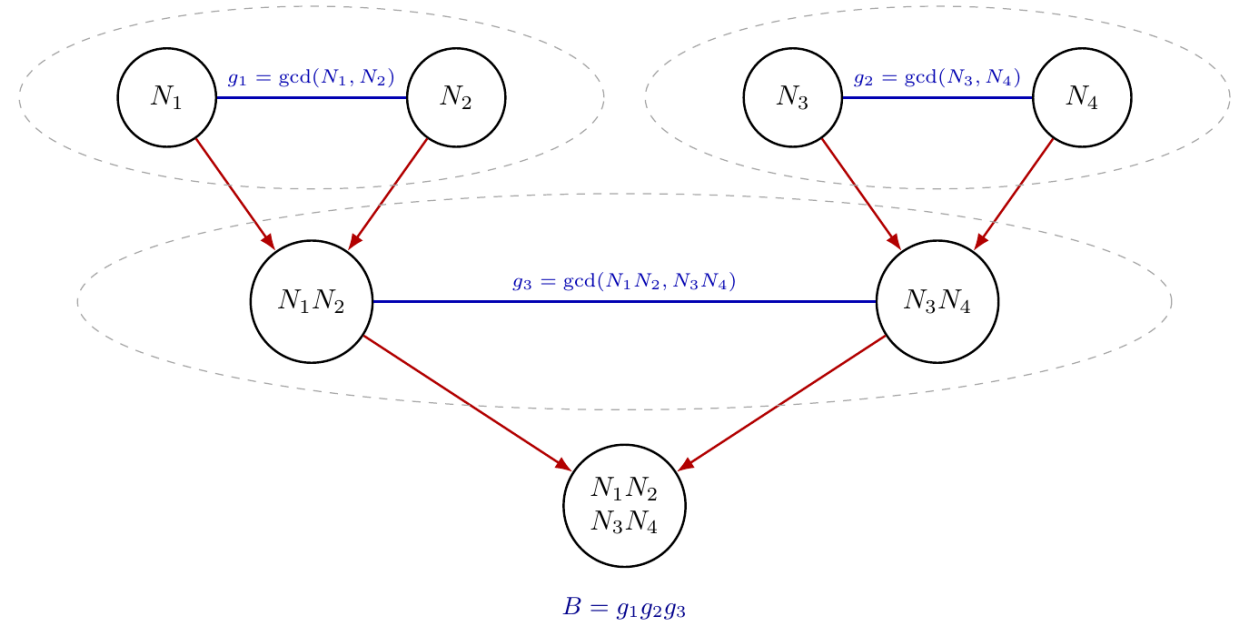
Este TFM

- Implementacion C++17/GMP
- Comparacion con Python
- Aplicacion a datos reales

El TFM toma el problema aplicado de Vage e incorpora la mejora algoritmica de Pelofske.

Binary Tree Batch GCD (Pelofske, 2024)

- Arbol binario de productos con gcd en linea.
- Agregacion de los GCD: $B = \text{producto de } g_j$.
- Comprobacion final: $\text{gcd}(N_i, B)$.
- No construye el remainder tree.
- Solo $2M - 1$ operaciones de gcd.



Objetivos del Trabajo de Fin de Master

- 1 Estudiar la deteccion de factores primos compartidos en RSA.
- 2 Revisar el batch GCD clasico y el Binary Tree Batch GCD.
- 3 Implementar Binary Tree Batch GCD en C++17/GMP.
- 4 Compararlo con las implementaciones Python de referencia.
- 5 Aplicarlo a modulos RSA reales extraidos de CT-archive.

Aportacion propia

Implementacion desarrollada

C++17

Lenguaje principal

GMP / GMP C++

Aritmetica multiprecision

OpenMP

Paralelizacion opcional

Python 3.12

Referencias de Pelofske

OpenSSL

Extraccion de modulos

ct-archive

Fuente de CT logs

Por que C++17/GMP? enteros muy grandes, eficiencia y control de memoria, con liberacion temprana de los nodos del arbol para contener el pico de RAM.

Flujo con datos reales de CT-archive



54.278

modulos unicos brutos

54.275

modulos RSA plausibles

Foco en certificados de issuer/ (CAs e intermediarias).

Experimentos sinteticos: disen

Tamanos de modulo

1024 y 2048 bits

Claves debiles (WEAK)

2, 100, 1000

Tamano del conjunto (N)

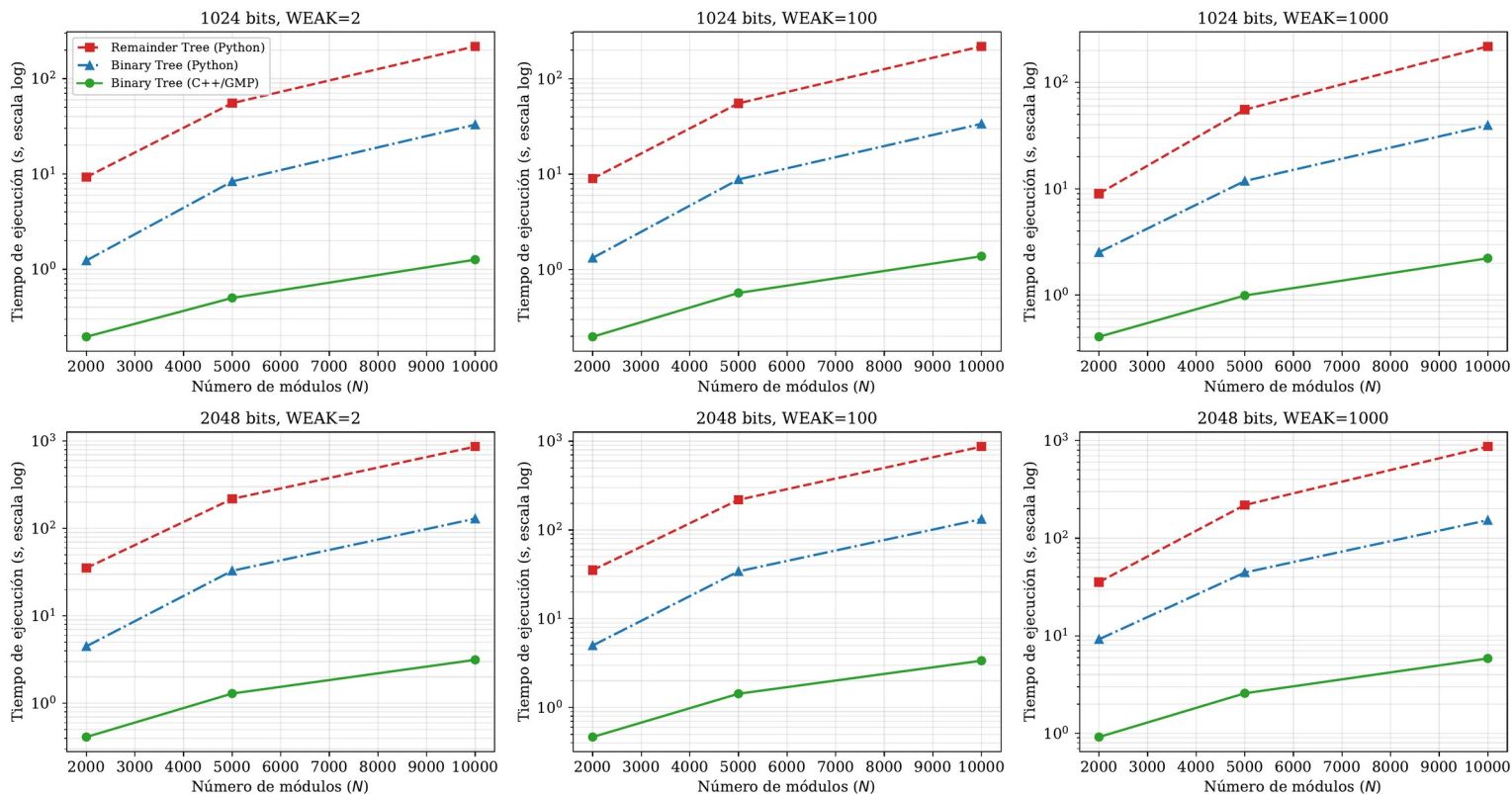
2000, 5000, 10000

Tres algoritmos sobre la misma entrada

- Remainder Tree Batch GCD (Python).
- Binary Tree Batch GCD (Python).
- Binary Tree Batch GCD (C++/GMP), propia.

Permite separar dos efectos: mejora algoritmica (Python vs Python) y mejora de implementacion (C++ vs Python).

Validacion, tiempo y memoria



Tiempo de ejecución (escala log)

Validacion

Las 3 implementaciones detectan los mismos módulos vulnerables.

Algoritmo

Binary Tree (Py) ~ 6,1x sobre el clasico.

Implementacion

C++/GMP: 116,5x medio (max 275,3x) sobre el clasico.

Memoria

Pico 57,9 MB: -55,0% vs clasico, -38,7% vs Binary Py.

Datos reales, conclusiones y limitaciones

54.278

modulos unicos brutos

54.275

modulos plausibles

0

factores compartidos no
triviales

0

claves vulnerables

~ 23,8 s

tiempo total del analisis

Conclusion (prudente)

- No se hallaron factores compartidos en este subconjunto...
- ...pero no permite extrapolar a todos los CT logs ni a todo Internet.

Limitaciones y trabajo futuro

- Conjunto real acotado (issuer/); parrilla sintetica reducida.
- Futuro: mas logs y certificados hoja, ejecucion por lotes, distribucion y mas memoria.

- **Material de apoyo**

Diapositivas de respaldo para preguntas del tribunal

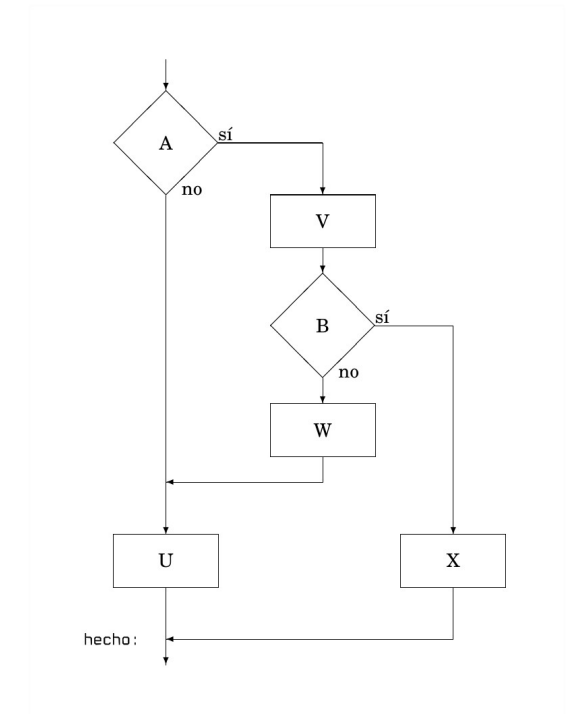
Product/Remainder Tree vs. Binary Tree Batch GCD

Aspecto	Clasico (product+remainder)	Binary Tree Batch GCD
Estructura	Product tree + remainder tree	Solo arbol binario de productos
Remainder tree	Se construye explicitamente	No se construye
Acumulacion	Residuos descendentes	GCD en linea -> $B = \text{producto de } g_i$
Operaciones gcd	- (via residuos)	$2M - 1$
Complejidad	Mismo orden asintotico	Mismo orden, menor coste practico

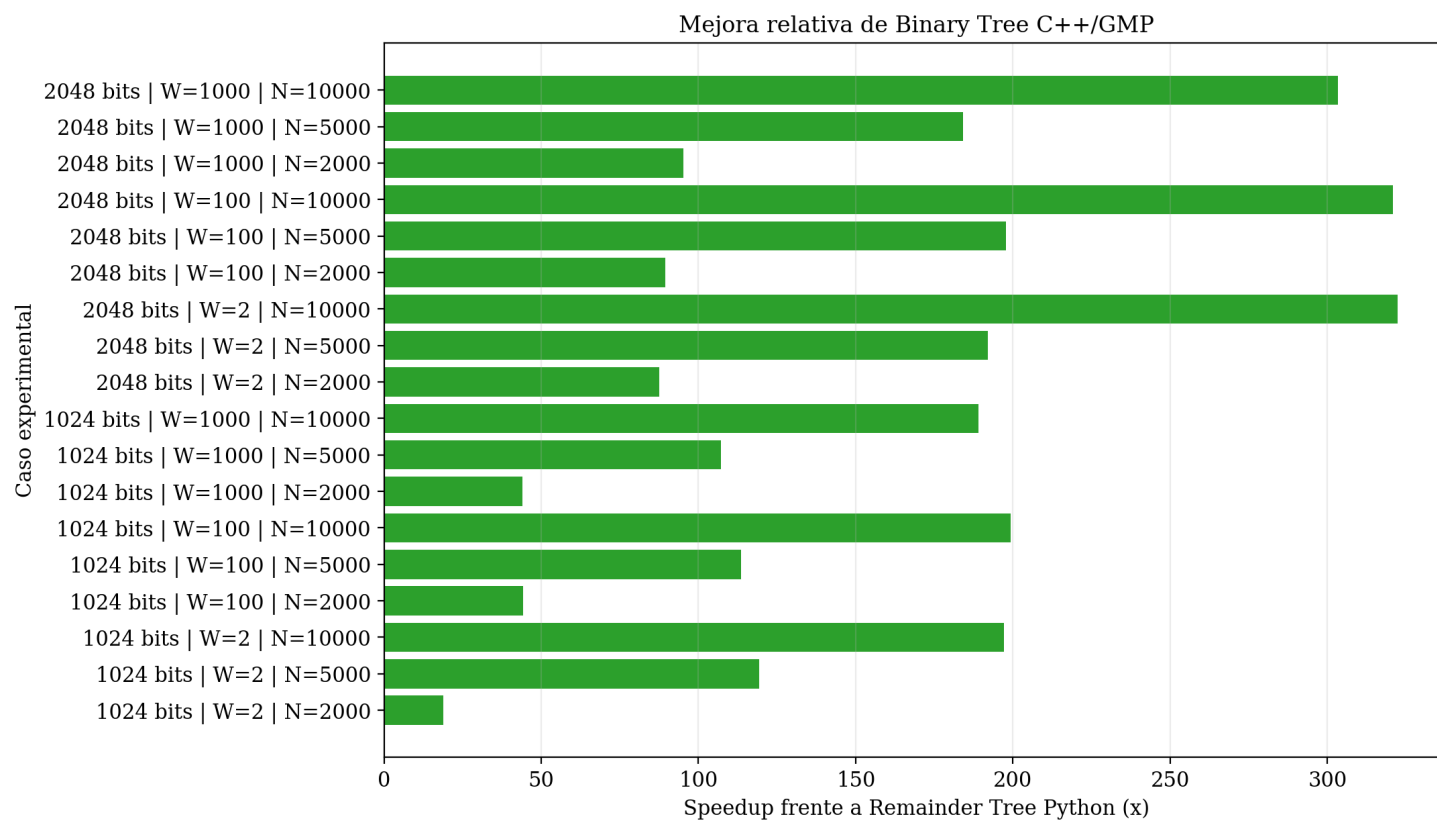
Idea central: ambos evitan la comparacion par a par; el binario suprime el remainder tree.

Por que '0 vulnerabilidades' tambien es un resultado?

- Confirma que el flujo completo funciona de extremo a extremo sobre datos reales.
- Aporta evidencia empirica acotada: en este subconjunto no hay factores compartidos.
- Es coherente con la teoria: en condiciones normales, compartir un primo es rarísimo.
- Acota el alcance: no demuestra ausencia de vulnerabilidades en todos los CT logs.
- Define una linea base reproducible para auditar conjuntos mayores en el futuro.



Cuanto mejora el algoritmo y cuanto C++/GMP?



~ 6,1x

mejora del ALGORITMO

Binary Tree (Py) vs. clasico (Py)

18,7x (max 41,3x)

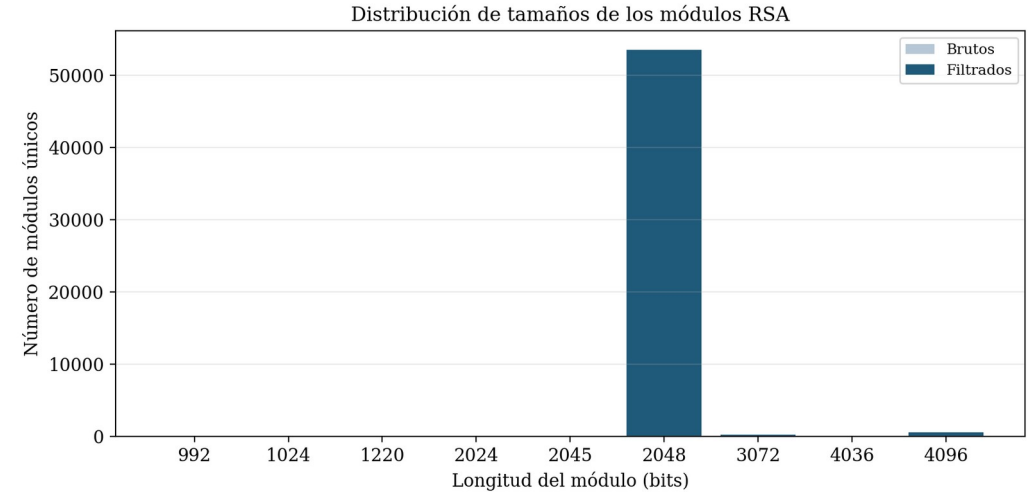
mejora de IMPLEMENTACION

C++/GMP vs. Binary Tree (Py)

Combinado vs. clasico: 116,5x (max 275,3x)

Que es un 'modulo plausible'?

- Validacion basica antes del ataque: descartar lo que no puede ser un modulo RSA.
- Se descartan modulos pares y los de longitud por debajo del umbral.
- De 54.278 brutos -> 54.275 plausibles (3 descartados: 2 pares y 1 demasiado corto).
- Mayoria de claves RSA-2048 (53.539); tambien 1024, 3072 y 4096 bits.



Como escalar el trabajo a mas CT logs

- Incorporar mas logs y los certificados hoja de las entradas completas.
- Ejecucion por lotes (batches) y cruce entre bloques, como en el metodo clasico.
- Paralelizar las fases aun secuenciales (agregacion de B).
- Distribucion y entornos con mas memoria para acercarse a la escala de la literatura.
- Automatizacion incremental de descarga, extraccion y auditoria.

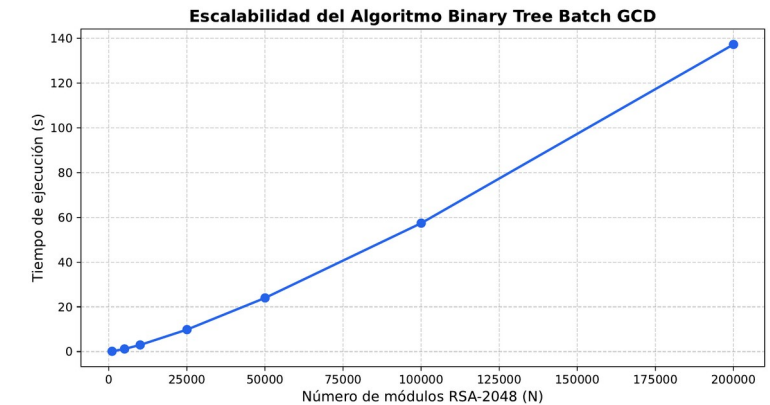


Tabla completa de resultados sinteticos

Bits	WEAK	N	C++/GMP (s)	Binary Py (s)	Remainder Py (s)	Speedup C++
1024	2	2000	0,196	1,236	9,266	47,3x
1024	2	5000	0,501	8,353	55,321	110,4x
1024	2	10000	1,262	32,896	218,123	172,8x
1024	100	2000	0,198	1,327	9,012	45,5x
1024	100	5000	0,570	8,815	55,288	97,0x
1024	100	10000	1,381	33,721	218,579	158,3x
1024	1000	2000	0,406	2,530	9,001	22,2x
1024	1000	5000	0,992	11,834	55,418	55,9x
1024	1000	10000	2,220	39,464	218,174	98,3x
2048	2	2000	0,411	4,486	35,272	85,8x
2048	2	5000	1,292	32,790	217,923	168,7x
2048	2	10000	3,139	129,501	864,321	275,3x
2048	100	2000	0,465	4,993	35,315	75,9x
2048	100	5000	1,429	34,203	218,786	153,1x
2048	100	10000	3,368	132,484	870,017	258,3x
2048	1000	2000	0,917	9,201	35,440	38,6x
2048	1000	5000	2,573	44,440	218,372	84,9x
2048	1000	10000	5,851	152,803	866,819	148,1x